



(<http://www.smlcodes.com/>)



Java (<http://www.smlcodes.com/category/java/>)

Web Development (<http://www.smlcodes.com/category/web-development/>)

Databases (<http://www.smlcodes.com/category/databases/>)

Errors (<http://www.smlcodes.com/category/errors/>)

Tutorials (<http://www.smlcodes.com/category/onepage-tutorial/>)

Tools (<http://www.smlcodes.com/category/tools/>)

Books & Downloads (<http://www.smlcodes.com/category/books-downloads/>)

SQL Basics for Java Developers

Posted by SmlCodes (<http://www.smlcodes.com/author/satyakavetigmail-com/>)

I.Introduction to SQL

SQL is a standard language for accessing and manipulating databases

RDBMS

- RDBMS stands for Relational Database Management System.
- RDBMS is the basis for SQL, and for all modern database systems such as MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access.
- The data in RDBMS is stored in database objects called tables.
- A table is a collection of related data entries and it consists of columns and rows

1. **Database Tables:** A database most often contains one or more tables. Each table is identified by a name (e.g. "Customers" or "Orders"). Tables contain records (rows) with data
2. **SQL Statements** Most of the actions you need to perform on a database are done with SQL statements. SQL keywords are **NOT case sensitive**: **select** is the same as **SELECT**

1.1 SQL Operations

Some of the Most Important SQL Commands

1. CREATE DATABASE – creates a new database

```
CREATE DATABASE jdbc
```

2. DROP DATABASE – modifies a database

```
DROP DATABASE jdbc
```

3. CREATE TABLE – creates a new table

```
1 CREATE TABLE `student` (  
2  
3 `sno` INT NOT NULL,  
4  
5 `name` VARCHAR(50) NULL,  
6  
7 `address` VARCHAR(50) NULL,  
8  
9 PRIMARY KEY (`sno`)  
10  
11 );
```

4. ALTER TABLE – modifies a table

```
1 ALTER TABLE `student` ADD COLUMN `city` INT(11) NOT ;
```

5. DROP TABLE – deletes a table

```
1 DROP TABLE `student`;
```

6. CREATE INDEX – creates an index (search key)

```
1 ALTER TABLE `student` ADD UNIQUE INDEX `city` (`city`);
```

7. DROP INDEX – deletes an index

```
1 ALTER TABLE `student` DELETE UNIQUE INDEX `city` (`city`);
```

8. INSERT INTO – inserts new data into a database

```
1 INSERT INTO `student` (`name`, `address`) VALUES ('Ravi', 'HYD');
```

9. **SELECT** – extracts data from a database

10. **UPDATE** – updates data in a database

11. **DELETE** – deletes data from a database

Select Operations

SELECT * FROM student ;

sno	name	address
101	Satya	HYD
102	Ravi	HYD
103	Surya	VIJ
104	Rakesh	CHENNAI
105	Madhu	GUN
106	Krishna	HYD
107	Satya	VIJ

SELECT s.sno, s.name FROM student s;

student (2×4)	
sno	name
101	Satya
102	Ravi
103	Surya
104	Rakesh

SELECT * FROM student s WHERE s.sno =104;

student (3×1)		
sno	name	address
104	Rakesh	CHENNAI

SELECT * FROM student s WHERE s.name="Satya" AND s.address="HYD";

sno	name	address
101	Satya	HYD

```
SELECT * FROM student s WHERE s.name="Satya" OR s.address="HYD";
```

sno	name	address
101	Satya	HYD
102	Ravi	HYD
106	Krishna	HYD
107	Satya	VIJ

```
SELECT * FROM student s WHERE s.name like '%a';
```

- **like** '%a' –Ending with 'a'
- **like** 'a%' – Starting with 'a'
- **like** '%a%' –Contains with 'a'
- **like** '%' –Contains any

```
SELECT * FROM student s WHERE s.address IN ('HYD', 'VIJ');
```

IN operator allows you to apply multiple where conditions

sno	name	address
101	Satya	HYD
102	Ravi	HYD
103	Surya	VIJ
106	Krishna	HYD
107	Satya	VIJ

```
SELECT * FROM student s WHERE s.sno BETWEEN 103 AND 106
```

(Mostly used in date comparision)

sno	name	address
103	Surya	VIJ
104	Rakesh	CHENNAI
105	Madhu	GUN
106	Krishna	HYD

SELECT DISTINCT s.address FROM student s;

address
HYD
VIJ
CHENNAI
GUN

SELECT s.name AS 'Student_Name', s.address AS 'CITY' FROM student s

sno	name	address
104	Rakesh	CHENNAI
105	Madhu	GUN
101	Satya	HYD
102	Ravi	HYD
106	Krishna	HYD
103	Surya	VIJ
107	Satya	VIJ

SELECT * FROM student s ORDER BY s.address asc; (default)

sno	name	address
103	Surya	VIJ
107	Satya	VIJ
101	Satya	HYD
102	Ravi	HYD
106	Krishna	HYD
105	Madhu	GUN
104	Rakesh	CHENNAI

```

1 SELECT * FROM `student` s ORDER BY s.address desc;
2
3 UPDATE student SET `name`='Vishnu' WHERE `sno`=107 AND `name`='Satya' AND `address`='VIJ' LIMIT 1;
4
5 DELETE FROM `mydb`.`student` WHERE `sno`=107 AND `name`='Vishnu' AND `address`='VIJ' LIMIT 1;

```

1.2 JOINS

An SQL JOIN clause is used to combine rows from two or more tables, based on a common field between them.

Rules

1. At least one common column must be present between two tables
2. PRIMARY_KEY of one table as FOREIGN_KEY of another table is recommended.

PRIMARY KEY Constraint

- The PRIMARY KEY constraint uniquely identifies each record in a database table.
- Primary keys must contain UNIQUE values.
- A primary key column cannot contain NULL values.
- Table can have **only ONE primary key**.

FOREIGN KEY Constraint

- A FOREIGN KEY in one table points to a PRIMARY KEY in another table.

CASCADE will propagate the change when the parent changes. (If you delete a row, rows in constrained tables that reference that row will also be deleted, etc.)

SET NULL sets the column value to NULL when a parent row goes away.

RESTRICT causes the attempted DELETE of a parent row to fail.

```
1 CREATE TABLE `customer` (  
2  
3 `cid` INT NOT NULL,  
4 `name` VARCHAR(50) NULL,  
5 `addr` VARCHAR(50) NULL,  
6 PRIMARY KEY (`cid`)  
7  
8 ) ;
```

```

1 CREATE TABLE `order` (
2
3 `ord_id` INT NOT NULL,
4
5 `ord_item` VARCHAR(50) NULL,
6
7 `ord_date` VARCHAR(50) NULL,
8
9 `cid` INT NULL,
10
11 PRIMARY KEY (`ord_id`),
12
13 CONSTRAINT `FK__customer` FOREIGN KEY (`cid`) REFERENCES `customer` (`cid`) ON UPDATE CASCADE
14
15 )
16
17 COLLATE='utf8_general_ci'
18
19 ENGINE=InnoDB;

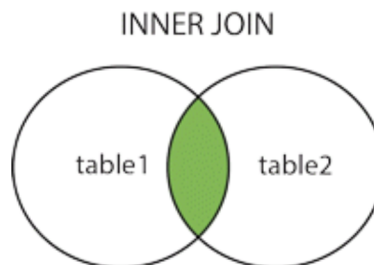
```

	name	addr	ord_id	ord_item	ord_date	cid
101	Satya	HYD	(NULL)	(NULL)	(NULL)	(NULL)
102	Ravi	VIJ	2005	COMPUTER	23-FEB-16	102
102	Ravi	VIJ	2007	CAR	20-JAN-16	102
103	RAKESH	CHENNEI	2001	TV	20-JAN-16	103
103	RAKESH	CHENNEI	2003	LAPTOP	20-MAR-16	103
104	Surya	BANG	(NULL)	(NULL)	(NULL)	(NULL)

We have 4 types of Joins

1. INNER JOIN:

Returns all rows when there is at least one match in BOTH tables

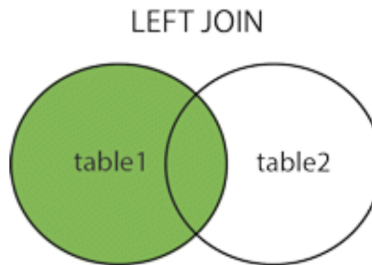


SELECT * FROM customer c **INNER JOIN** order o **ON** c.cid=o.cid

cid	name	addr	ord_id	ord_item	ord_date	cid
103	RAKESH	CHENNEI	2001	TV	20-JAN-16	103
103	RAKESH	CHENNEI	2003	LAPTOP	20-MAR-16	103
102	Ravi	VIJ	2005	COMPUTER	23-FEB-16	102
102	Ravi	VIJ	2007	CAR	20-JAN-16	102

2. LEFT JOIN:

Return all rows from the left table, and the matched rows from the right table

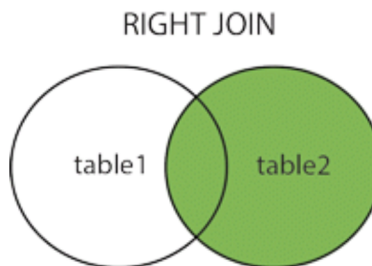


SELECT * FROM customer c **LEFT JOIN** order o **ON** c.cid=o.cid

	name	addr	ord_id	ord_item	ord_date	cid
101	Satya	HYD	(NULL)	(NULL)	(NULL)	(NULL)
102	Ravi	VIJ	2005	COMPUTER	23-FEB-16	102
102	Ravi	VIJ	2007	CAR	20-JAN-16	102
103	RAKESH	CHENNEI	2001	TV	20-JAN-16	103
103	RAKESH	CHENNEI	2003	LAPTOP	20-MAR-16	103
104	Surya	BANG	(NULL)	(NULL)	(NULL)	(NULL)

3. RIGHT JOIN:

Return all rows from the right table, and the matched rows from the left table

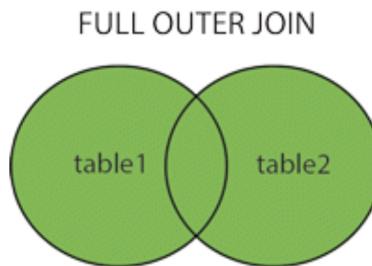


SELECT * FROM customer c **RIGHT JOIN** order o **ON** c.cid=o.cid

cid	name	addr	ord_id	ord_item	ord_date	cid
102	Ravi	VIJ	2005	COMPUTER	23-FEB-16	102
102	Ravi	VIJ	2007	CAR	20-JAN-16	102
103	RAKESH	CHENNEI	2001	TV	20-JAN-16	103
103	RAKESH	CHENNEI	2003	LAPTOP	20-MAR-16	103

4. FULL JOIN:

Return **all rows** when there is a match in **ONE** of the tables



```
SELECT * FROM customer c FULL OUTER JOIN order o ON c.cid=o.cid
```

In MySQL we don't have Full Outer Join, for this we have to use Unions

UNION Operator

SQL UNION operator **combines the result of two or more SELECT statements**

```
1 SELECT * FROM customer c WHERE c.cid>102
2 UNION
3 SELECT * FROM customer c
```

cid	name	addr
103	RAKESH	CHENNEI
104	Surya	BANG
101	Satya	HYD
102	Ravi	VIJ

- The UNION operator selects only distinct values by default.
- To allow duplicate values, to get duplicates also use **UNION ALL**
- Column name must be equal in **TWO** tables

SELECT INTO

SELECT INTO statement **copies data from one table** and inserts it into a **new table**.

```
SELECT * INTO old_customer FROM customer;      (mysql not supporting)
```

INSERT INTO SELECT

It selects data from one table and inserts it into an **existing table**

Constraints

- NOT NULL – Indicates that a column cannot store NULL value
- UNIQUE – Ensures that each row for a column must have a unique value
- PRIMARY KEY – A combination of a NOT NULL and UNIQUE
- FOREIGN KEY – Ensure the referential integrity of the data in one table to match values in another table
- CHECK – Ensures that the value in a column meets a specific condition
- DEFAULT – Specifies a default value for a column

Views

View is a **virtual table based on the result-set of an SQL statement**.

- A view contains rows and columns, just like a real table.
- The fields in a view are fields from one or more real tables in the database

Syntax :

```
1 CREATE OR REPLACE VIEW
2 view_name
3 AS
4
5 SELECT column_name(s)
6 FROM table_name
7 WHERE condition
```

Example

```
1 CREATE OR REPLACE VIEW [Current Product List] AS
2 SELECT ProductID,ProductName,Category
3 FROM Products
4 WHERE Discontinued=No
```

We can perform INSERT, DELETE, UPDATE Operations as normal as Table operations

Date Functions

The following table lists the most important built-in date functions in MySQL:

Function	Description
NOW() (http://www.w3schools.com/sql/func_now.asp)	Returns the current date and time
CURDATE() (http://www.w3schools.com/sql/func_curdate.asp)	Returns the current date
CURTIME() (http://www.w3schools.com/sql/func_curtime.asp)	Returns the current time
DATE() (http://www.w3schools.com/sql/func_date.asp)	Extracts the date part of a date or date/time expression
EXTRACT() (http://www.w3schools.com/sql/func_extract.asp)	Returns a single part of a date/time
DATE_ADD() (http://www.w3schools.com/sql/func_date_add.asp)	Adds a specified time interval to a date
DATE_SUB() (http://www.w3schools.com/sql/func_date_sub.asp)	Subtracts a specified time interval from a date
DATEDIFF() (http://www.w3schools.com/sql/func_datediff_mysql.asp)	Returns the number of days between two dates
DATE_FORMAT() (http://www.w3schools.com/sql/func_date_format.asp)	Displays date/time data in different formats

1.3 Sql Functions

Number functions

- **AVG()** – Returns the average value
- **COUNT()** – Returns the number of rows
- **FIRST()** – Returns the first value
- **LAST()** – Returns the last value
- **MAX()** – Returns the largest value
- **MIN()** – Returns the smallest value
- **SUM()** – Returns the sum

String functions

- **UCASE()** – Converts a field to upper case
- **LCASE()** – Converts a field to lower case
- **MID()** – Extract characters from a text field
- **LEN()** – Returns the length of a text field
- **ROUND()** – Rounds a numeric field to the number of decimals specified
- **NOW()** – Returns the current system date and time
- **FORMAT()** – Formats how a field is to be displayed

1.4 PL/SQL

PL/SQL stands for (Procedural Language/Structure Query Language). It is extension of SQL

2.1 Basics

1. Datatype

Below are supported Datatypes in PL/SQL which are supported in SQL.

	Data type	Syntax
1	Integer	INTEGER

2	Smallint	SMALLINT
3	Numeric	NUMERIC(P,S)
4	Real	REAL
5	Decimal	DECIMAL(P,S)
6	Float	FLOAT(P)
7	Character	CHAR(X)
8	Character varying	VARCHAR2(X)
9	Date	Date
10	Time	TIME

2. Variables

Variable Declaration

```
Variable_name datatype;
```

```
a number;
```

Variable Initialization

```
Variable_name datatype:=value;
```

```
a number=10;
```

3. Input Statement : to provide input value at run time by using operator (&).

4. Output Statement:

```
dbms_output.put_line ('Message' || variable);
```

```
dbms_output.put_line ('Sum: ' || c);
```

Basic Syntax

```
1 begin
2   dbms_output.put_line("Hello word");
3 end;
```

Very simple, in the place of {}, we use begin ... end;

Procedure

A pl-sql Procedure does not return any value. Procedure has two sections;

- **Declaration of the procedure:** Declaration of procedure always start with a keyword create ends with last **variable parameters**.
- **Body of the procedure:** Body of procedure starts with a keyword called **is or as** and ends with end statement.

Example

```
1 create or replace procedure p1(a in number, b out number, c out number, d out number)
2 is
3     begin;
4         select ename, sal, deptno, into b, c, d from emp where empno=a;
5     end;
```

Function

A PL/SQL Function is a self control block which is used to perform some specific task. **Function must always return a value**, but a procedure may or may not return a value.

```
1 create or replace function add(a number, b number)
2 return number
3 is
4 number;
5 begin;
6     a:=10;
7     b:=20;
8     c=a+b;
9 return c;
10 end;
```

Package is a collection of sub-programs that means function or procedure

```
1 create package package_name AS
2     procedure procedure_name(parameter);
3 end package_name;
```

Cursors

A **Cursors** is a temporary work area created in the system memory when a SQL statement is executed. It is a temporary memory which is used to fetch more than one record at a time from existing table.

Implicit cursor cursor is perform by the system internally

Explicit cursor This type of cursor is performed by the user manually or programmatically

Steps to perform cursor

Steps	Syntax
Declare the cursor	open cursor_name;
Open the cursor	open cursor_name;
Fetch the record from the cursor	fetch cursor_name into variables;
Close the cursor	close cursor_name;

-

Declare the cursor

```
1 declare
2 a emp %rowtype;
3 cursor c is select * from emp where depno=&deptno;
4 begin
5 open c;
6 loop fetch c into a;
7 if c % found then
8 dbms_output.put_line(a.empno || ' ' a.ename || ' ' || a.sal);
9 else
10 exit;
11 end if;
12 end loop;
13 close c;
14 end;
```

Trigger

Trigger is a pl/sql block structure which is fired when a DML statements like Insert, Delete, Update is executed on a database table. A trigger is triggered automatically when an associated DML statement is executed

Purpose of Triggers

Triggers can be written for the following purposes:

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access Auditing
- Synchronous replication of tables
- Imposing security authorizations
- To avoid invalid transactions
- To generate the resulting data automatically.

Part of Trigger

A database trigger has 5 parts.

- Trigger timing
- Trigger event or statement
- Trigger level
- Trigger restriction
- Trigger body


```

1 CREATE [OR REPLACE ] TRIGGER trigger_name
2 {BEFORE | AFTER | INSTEAD OF }
3 {INSERT [OR] | UPDATE [OR] | DELETE}
4 [OF col_name]
5 ON table_name
6 [REFERENCING OLD AS o NEW AS n]
7 [FOR EACH ROW]
8 WHEN (condition)
9 BEGIN
10 --- sql statements
11 END;
```

Write a trigger to stop delete operation on emp_table

create or replace trigger mytrigger

```

1 create or replace trigger mytrigger
2 before
3 delete
4 on emp
5 begin
6 raise_application_error(-20000, 'sorry we can not delete any record from this table');
7 end;
```

-

References

- <http://www.w3schools.com/sql/> (<http://www.w3schools.com/sql/>)
- <http://www.sitesbay.com/plsql/> (<http://www.sitesbay.com/plsql/>)

Share with Friend

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=twitter&nb=1>)

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=facebook&nb=1>)

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=google-plus-1&nb=1>)

 (whatsapp://send?text=SQL%20Basics%20for%20Java%20Developers http%3A%2F%2Fwww.smlcodes.com%2Fdatabases%2Fsql-basics-java-developers%2F)

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=pinterest&nb=1>)

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=linkedin&nb=1>)

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=skype&nb=1>)

 (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=tumblr&nb=1>)

✉ (<http://www.smlcodes.com/databases/sql-basics-java-developers/?share=email&nb=1>)

🖨 (<http://www.smlcodes.com/databases/sql-basics-java-developers/#print>)

Like this:

★ Like

Be the first to like this.

Related

HTML parser in Java –jsoup
Tutorial Onepage
(<http://www.smlcodes.com/java/html-parser-java-jsoup-tutorial-onepage/>)
July 22, 2016
In "HTML"

IV. Java Exception Handling
(<http://www.smlcodes.com/core-java-complete/iv-java-exception-handling/>)
September 24, 2016
In "core-java-complete"

V. Java UI (Applets/Swings)
(<http://www.smlcodes.com/core-java-complete/java-applets-swings/>)
September 24, 2016
In "core-java-complete"

📁 Databases (<http://www.smlcodes.com/category/databases/>), Onepage Tutorial (<http://www.smlcodes.com/category/onepage-tutorial/>)

🔖 Tutorial (<http://www.smlcodes.com/tag/tutorial/>)

← So Called CORE JAVA – Satya Kaveti's eBook Download (<http://www.smlcodes.com/onepage-tutorial/socalled-core-java-satya-kavetis-ebook-download/>)

Leave a Reply

Enter your comment here...

Search ...

Recent Posts

SQL Basics for Java Developers (<http://www.smlcodes.com/databases/sql-basics-java-developers/>)

So Called CORE JAVA – Satya Kaveti's eBook Download (<http://www.smlcodes.com/onepage-tutorial/socalled-core-java-satya-kavetis-ebook-download/>)

[tutorial/socalled-core-java-satya-kavetis-ebook-download/](#))

XII Design Patterns & EJB Just Introduction (<http://www.smlcodes.com/core-java-complete/xii-design-patterns-ejb-just-introduction/>)

XI. Java Features 1.x to Till Now (<http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/>)

X. Java Networking (java.net. *) (<http://www.smlcodes.com/core-java-complete/x-java-networking-java-net/>)

Archives

September 2016 (<http://www.smlcodes.com/2016/09/>)

August 2016 (<http://www.smlcodes.com/2016/08/>)

July 2016 (<http://www.smlcodes.com/2016/07/>)

Categories

Books & Downloads (<http://www.smlcodes.com/category/books-downloads/>)

Carrier Tips (<http://www.smlcodes.com/category/carrier-tips/>)

core-java-complete (<http://www.smlcodes.com/category/core-java-complete/>)

Databases (<http://www.smlcodes.com/category/databases/>)

DevOps (<http://www.smlcodes.com/category/devops/>)

Errors (<http://www.smlcodes.com/category/errors/>)

HTML (<http://www.smlcodes.com/category/html/>)

jar (<http://www.smlcodes.com/category/jar/>)

Java (<http://www.smlcodes.com/category/java/>)

JSON (<http://www.smlcodes.com/category/json/>)

[jsoup \(http://www.smlcodes.com/category/jsoup/\)](http://www.smlcodes.com/category/jsoup/)

[Onepage Tutorial \(http://www.smlcodes.com/category/onepage-tutorial/\)](http://www.smlcodes.com/category/onepage-tutorial/)

[PHP \(http://www.smlcodes.com/category/php/\)](http://www.smlcodes.com/category/php/)

[Servers \(http://www.smlcodes.com/category/servers/\)](http://www.smlcodes.com/category/servers/)

[Tools \(http://www.smlcodes.com/category/tools/\)](http://www.smlcodes.com/category/tools/)

[XML \(http://www.smlcodes.com/category/xml/\)](http://www.smlcodes.com/category/xml/)

Pages

Archives

Meta

About

(<http://www.smlcodes.com/about/>)

Contact US

(<http://www.smlcodes.com/contact/>)

Privacy Policy

(<http://www.smlcodes.com/privacy/>)

Terms and
Conditions

(<http://www.smlcodes.com/terms/>)

September
2016

(<http://www.smlcodes.com/2016/09/>)

August 2016

(<http://www.smlcodes.com/2016/08/>)

July 2016

(<http://www.smlcodes.com/2016/07/>)

Log in

(<http://www.smlcodes.com/wp-login.php>)

© 2016 smlcodes.com

About Us (/about) Terms (/terms) Privacy policy (/privacy) Contact


(ht
tps
:
//
w
ww
.g
+.in
(ht
(ht
.yo
ut
tps
:
//
ub
w
ww
.fa
pl
e.c
ce
us.
ww
om
bo
go
.lin
/c
ok.
og
ke
ha
co
le.
din
nn
m
co
.co
el

/S
m
m
/U
(ht
ml
/1
/in
CO
tps
Co
11
/s
0Jj
:/t
de
37
ml
Za
wit
s-
38
-
8B
ter
21
03
co
cR
.co
69
45
de
vc
m
71
63
s-
3r
/s
27
60
14
Te
ml
53
30
47
0B
co
64
39
4b
L9
de
32
61
12
HA
(/contact) | s) 0/) /) 2))